

Pokélearn

Hannah Garcia, Adam Ward, and Bitna White

5/1/2026

Problem Statement

The Pokélearn project developed a decision-making model for the Pokémon Showdown application that makes choices as close to optimal as possible in a given battle scenario. The agent used rule-based heuristics and a policy-value neural network using a multi-layer perceptron with opponent move prediction through shallow search. For the UI facing end of the project, a desktop app was created with React and Electron. The UI mimics a Pokédex recommender that tells the user which move to select at a given turn, with options to view the model's reasoning.

All training was conducted in the Generation 1 Random Battle (Gen1 RandBat) format of the game to create a consistent set of variables and a fair training environment. This battle format limits the Pokémon available to those that existed in the first game - hence, first generation - while giving the two opposing players a random, battle-optimized team of Pokémon. The model will have to leverage multiple different interacting factors, such as both players' Pokémon health, damage, defensive stats, and teams, in order to provide the most optimal decision leading to the best result in any given scenario. Ultimately, the success of this model will be primarily determined by its win rate against other users and models.

Methodology

Overview

Pokélearn was designed to integrate real time battle data from the Pokémon Showdown¹ platform to apply mechanics evaluation and heuristic reasoning to determine the best course of action for the player to follow. Pokémon Showdown is a web-based Pokémon battle simulator widely used for practicing and competing in Pokémon battles through ranked battles and online tournaments. The matchmaking is done in real-time and pits two players against each other in a battle, which is restricted based on the selected ruleset.

One such restriction is based on generation, which is a method to separate groups of Pokémon based on the main line Pokémon game they were released with. Another restriction is based on the tier of a Pokémon, which is a ranking provided by the online community called Smogon¹ that determines the power level of a Pokémon while running specific strategies, in their respective generations. Finally, there is also the doubles or singles format, which determines how many active Pokémon from the player's party are in the field at once.

For the purpose of our model's training and usage, we selected the Gen1 RandBat format, as it features the 151 original Pokémon in Generation 1, reduced from the 1025 available in the Gen 9 format, which we determined was a more suitable dataset based on the amount of time available for training. The random battles feature additionally populates both the user's and the opponent's parties with random Pokémon with competitive movesets, which allows the model to be exposed to various situations and strategies and removes the requirement to create viable competitive parties for the model to use. An important distinction to make is that although the format features the original 151 Pokémon from Generation 1, several key gameplay mechanics are still retained from future generations.

¹ ("Formats | RB", n.d.)

Pokémon Mechanics

One such gameplay mechanic is Pokémon typing. Each individual Pokémon is assigned either one or two of 15 official types, with distinct advantages and disadvantages within the game's mechanics². Later iterations of the game feature additional types, but are not included since the types are snapshotted from Generation 1.

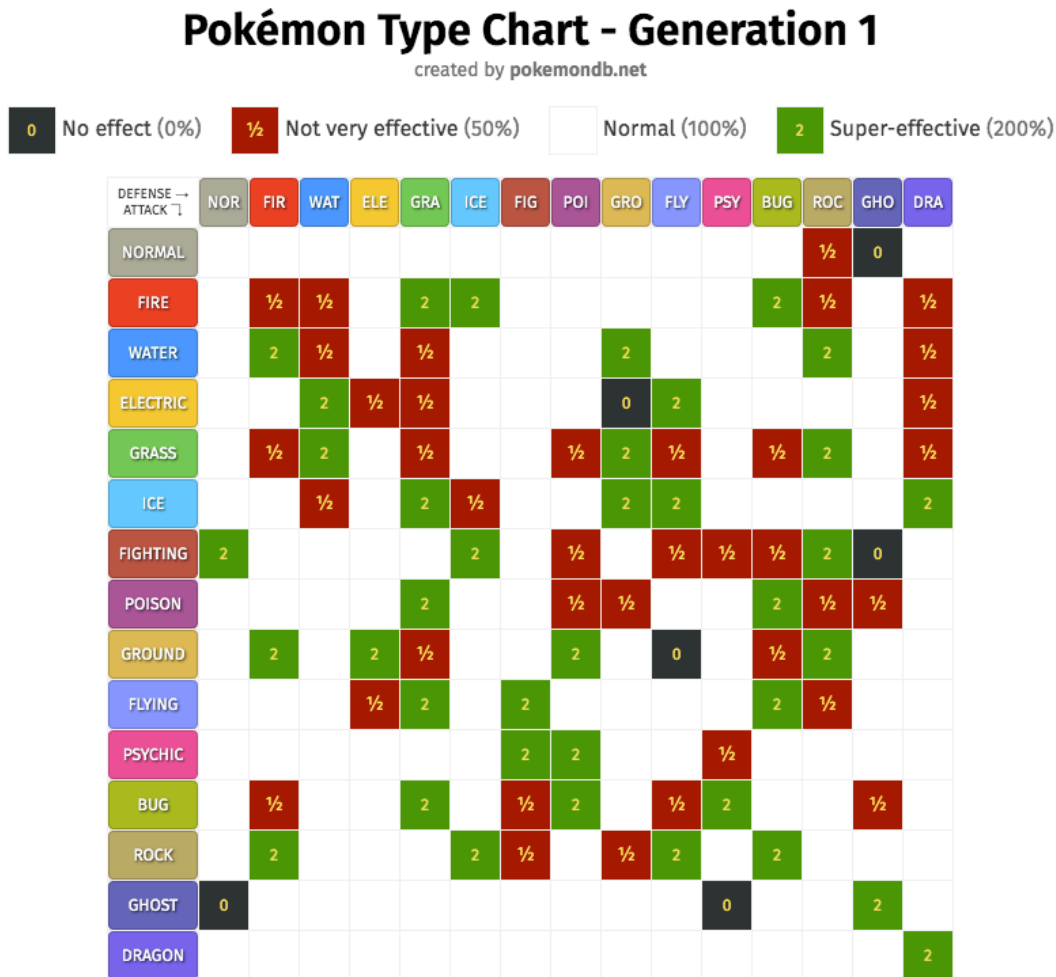


Figure 1: Pokémon Type Chart from Generation 1.

Figure 1 displays the offensive and defensive type chart used in the Gen1 RandBat format. Alongside types, each Pokémon has several other key unique stats that ultimately affect damage calculation. There are six stats total inherent to the Pokémon that make up its base stat total (BST). This includes health, also referred to as health points (HP), attack (ATK), special attack (SPATK), defense (DEF), and special defense (SPDEF). There are a few factors that affect a Pokémon's final stat values. Each individual Pokémon has base stats that are equal amongst each Pokémon of the same species. For example, one such Pokémon, Pikachu, will always have the same BST as another Pikachu. There are additional modifiers on top of base stats, one of which is stat growth. Detailed stat information and

² ("Pokémon type chart: strengths and weaknesses | Pokémon Database", n.d.)

calculations are available for each Pokémon online⁸ (“Stat - Bulbapedia, the community-driven Pokémon encyclopedia”, n.d.). For Pokémon showdown, unique stat modifiers known as effort values (EVs) and individual values (IVs) are set to the same value for each Pokémon regardless of species. However, Pokémon levels are adjusted to balance Pokémon strength based on winrate, which increase or decrease stats accordingly to balance the metagame.

Pokémon also individually carry four unique moves in their moveset. Each Pokémon has access to its own individual move pool, which is a set of unique moves available for each species to choose from. Pokémon Showdown assigns a competitively viable moveset for the Pokémon based on game balance and win rate. Each move in Pokémon is sorted into a unique category based on its effect in battle. Each move also has its own unique type, power, accuracy, and power points (PP), which indicate the number of times each move can be used. In summary, all of these factors come together to determine the damage and effects of moves. In Gen1 RandBats, only moves available in Generation 1 are viable to be used³.

When calculating final damage dealt, there is a specific equation used to determine the exact number. The damage dealt depends on a multitude of factors, including moves and terrain setup during battle, and both Pokémon’s stat values. Showdown uses damage calculation methods specifically for Generation 1 to determine damage. It’s important to note that final damage isn’t deterministic because a random variable is included in the damage calculation. This means that although there is a fixed range, it’s not possible to know exactly how much damage a move will do each hit.

In addition to the non-determinism of move damage, there are many factors that are hidden but need to be predicted or accounted for. At the start of battle, the only available information about the enemy is the first Pokémon in their party. As the battle progresses, any moves they use or additional Pokémon sent out are information that needs to be learned over the course of the battle. As a result, in order to be successful, the model needs to learn patterns in movesets and player behavior.

Implementation

The main pipeline of the system contains the following steps:

1. A poke-env (library) agent receives a live Showdown battle object when a move is requested.
2. The state parser converts the battle into our internal State object.
3. The decision engine is called with the created State and uses the mechanics engine and its heuristics, search, and the learning model, to determine the most optimal move.
4. The system returns the top-ranked move suggestions with reasoning.
5. The UI control panel displays the suggestions to the user.
6. The user confirms which move to use, and the move is passed back to the poke-env agent and into Showdown.

Showdown Integration (poke-env)

In order to read battle data from Showdown, the library poke-env is used⁴. The use of poke-env assisted in our project for configuring and interacting with Pokémon Showdown servers, as it provided a straightforward and seamless API to communicate with servers and perform basic tasks. This includes connecting to account and server configuration, collecting battle and state information, and providing relevant game mechanic tools - such as built-in damage calculators.

³ (“Pokémon moves from Generation 1 | Pokémon Database”, n.d.)

⁴ (“Poke-Env”, n.d.)

State Parser

The state parser converts live poke-env battle objects into our internal State for other modules to use for calculations and decisions. In other words, the state parser determines what the current game state is. The state parser's key data classes consist of PokemonSnapshot, LegalMove, and State.

PokemonSnapshot is used for both active and revealed team members and stores the species, current HP, status condition, stat modifiers or boosts, types, known moves, knowledge state (seen or not seen), whether the Pokémon has fainted or not, and specific poke-env identifier attributes. Each move a Pokémon can perform is stored in the LegalMove data class, which contains its action ID, move name, move type, base power, damage class (physical vs. status), who the move targets (one opponent vs. multiple opponents), accuracy, move priority, current and max PP, if the move is a switch, and relevant poke-env attributes. The State data class consists of the previous dataclasses, storing a PokemonSnapshot of the friendly team's current active Pokémon, a PokemonSnapshot of the opponent team's current active Pokémon, a unique battle tag, the current turn number, whether a Pokémon was forced to switch, the type of request needed (ex: a move, a forced switch, etc.), the entire friendly team, the entire opponent team, the number of known friendly Pokémon, the number of known opponent Pokémon, all legal actions available during a turn, and relevant poke-env attributes.

Mechanics Engine

The mechanics engine replicates how much damage moves do to the current Pokémon, how much damage has been taken, status effects, and more. This is a physics engine that takes no part in decision making, but is an integral foundation for building the decision-making model. The mechanics engine was implemented using Pokémon Showdown's built-in damage logic, with communication being facilitated using poke-env.

Given a state and a candidate action, the mechanics engine estimates the outcome of that action using damage calculation logic consistent with Pokémon Showdown's Gen1 Randbat ruleset. Because damage in this format includes the random component, the engine necessitates expected values rather than deterministic outcomes. This includes the following: the expected damage to the opponent is calculated using poke-env's Gen 1-2 damage calculator for non-deterministic "rolls" (minimum roll for the smallest possible damage, maximum roll for the largest possible damage, and all possible rolls). Rolls are then used to determine the damage being taken. In addition, the expected damage to self, the KO probability to the opponent, and the KO probability to self are calculated similarly. The mechanics engine also determines if the friendly team acts first (based on priority and speed), calculates accuracy based on current status effects, and type effectiveness, which computes a move's type multiplier from the move type vs. the opponent's types.

These outputs allow the system to reason probabilistically about the consequences of actions while remaining faithful to the underlying game mechanics. The mechanics engine additionally serves as the foundation for both the heuristic evaluation and learning-based prediction, and makes sure that decisions are aligned with the reality of the game.

Decision Engine

On top of the mechanics engine is the decision engine, which decides what move to pick given a current state. Initially, we use a heuristic evaluation system and shallow search that provide the baseline policy for decision making, before moving to training via a Policy-Value network. The heuristic model assigns a score to each possible action based on features available from the mechanics engine, including:

the probability of immediately KO-ing an opponent, the probability of a move KO-ing ourselves, the expected damage dealt, expected damage taken, the move priority, type effectiveness, accuracy, fainted status, status moves and effects, and low HP thresholds. Some conditions give a high reward, such as guaranteeing a KO, while dangerous conditions are penalized. The heuristics system alone enables the agent to make good decisions in simple 1v1 scenarios without any prediction possibility and without needing learned behavior.

Since Pokémon battles have many possibilities that need to be accounted for, the agent additionally uses a shallow search procedure that explores possible future states. For each action candidate, the system simulates likely opponent responses and evaluates the resulting states over a limited depth. The search process uses the mechanics engine to approximate state transitions and applies a discounted evaluation of future outcomes, allowing the agent to balance immediate rewards with longer term consequences. Because exhaustive search is not possible computationally due to the amount of branching that would occur, the system restricts the exploration to plausible opponent actions and applies depth based discounting for efficiency and accuracy. The search component improves decision-making quality by allowing the agent to anticipate opponent behavior, properly predicting future moves rather than reacting to the specific current state.

Training

The primary learning component of Pokélearn is a policy value network that is implemented as a multi-layer perceptron (MLP)⁵. The network takes the encoded game state as input and produces two outputs. One is a policy distribution over possible actions, and the other is a scalar value estimate representing the probability of winning from the current state. The architecture contains several layers that are connected, and two output heads that correspond to the policy and value functions. This design allows the network to learn which actions are promising and how favorable a given state is. The policy output is used to prioritize some actions for a deeper evaluation, while the value output provides a learned approximation of long term success beyond the scope of the search.

The training of the neural network is conducted in multiple stages, beginning with supervised learning on data generated by the heuristic agent. During this phase, gameplay logs are collected, and the network is trained to imitate the heuristic policy while also learning how to predict game outcomes. The supervised training provides initialization for the model to replicate baseline decision making behavior before improving on it.

After the supervised training, the system enters self training in which the model generates new data by competing in simulated battles. These experiences are added to the training dataset, and the model is periodically retrained on the expanded data. Over successive iterations, the model refines its policy and value estimates to learn patterns that are beyond the limitations of heuristic rules. To ensure that models are improving, they are evaluated to determine if the win rate is actually increasing and to avoid regression in the training.

Ultimately, the policy network evaluates the current state and selects a subset of high probability actions, which are then passed into the search module, which simulates their outcomes using the mechanics engine while incorporating likely opponent responses. The resulting states are evaluated using the value network, and the final decision is chosen based on all these factors combined.

UI Control Panel

⁵ (Flaherty, Jimenez, and Abbasi 2023, 2)

The system presents its recommendations through a user interface built with React and Electron. The interface displays the top-ranked moves along with supporting reasoning, allowing a human user to make the final decision and execute the selected action within the battle. Rather than working autonomously, the agent is designed as a decision support tool that allows a human player to review and select from the recommended actions. This design choice reflects the project’s goal of creating an interpretable and interactive AI system, and aims to be an assistant to learning how to play the game rather than a cheating tool. As a result, we implemented the ability to see reasoning behind why certain moves are recommended, and encourage critical thinking as the user to determine if other moves may be more beneficial than what the model is suggesting as well.

Evaluation

Evaluation for Pokélearn is based on the win-rate over a specific baseline. For example, the first few trained models were trained across 100 battles total before evaluation, while later models were trained across 300 battles total before evaluation. We also keep track of training behavior through battle logs, checkpoints, and metrics. Battle logs consist of the battle tag, turn number, numeric version of a state, legal actions during the state, the chosen actions, outcome value (the numerical value of how the decision turns out for us), policy target (the correct answer for what move to choose), and optional extra data for the decision.

Results

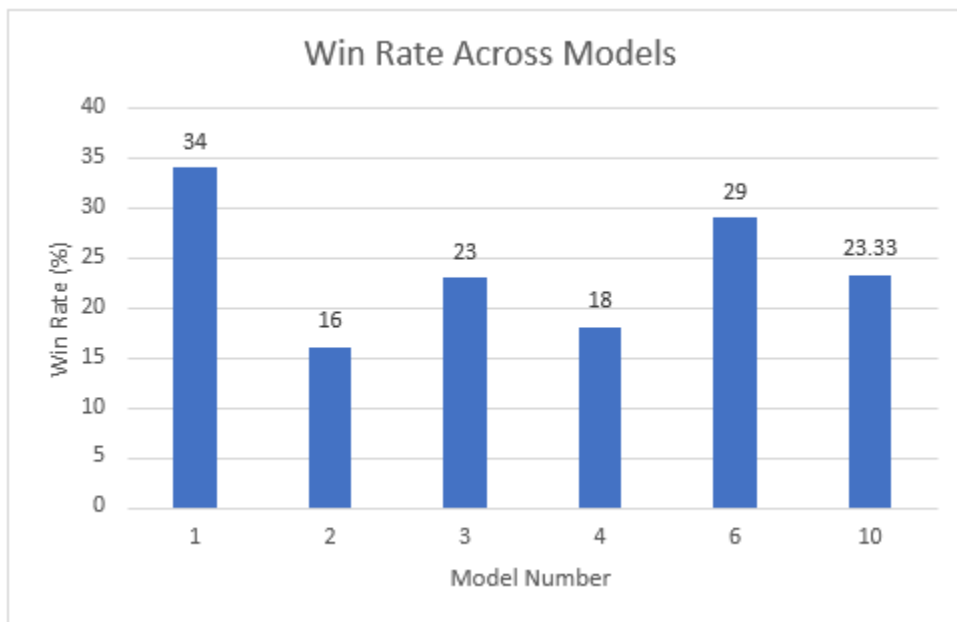


Figure 2: Visualization of Win Rate of different model iterations.

Figure 2 visualizes the win rate of successive model iterations, which shows the model is able to achieve win rates of about 30%. Compared to the performance of LLMs as game players, it achieves a notably much higher win rate, such as when evaluating against a random selection bot. which achieves only a 1.2% winrate, or MaxPower, which achieves a winrate of 10.40%⁶. The most comparable performance of LLMs in battles was GPT-4, which achieved a win rate of 26%.

⁶ (Hu, Huang, and Liu 2024, 5.1)

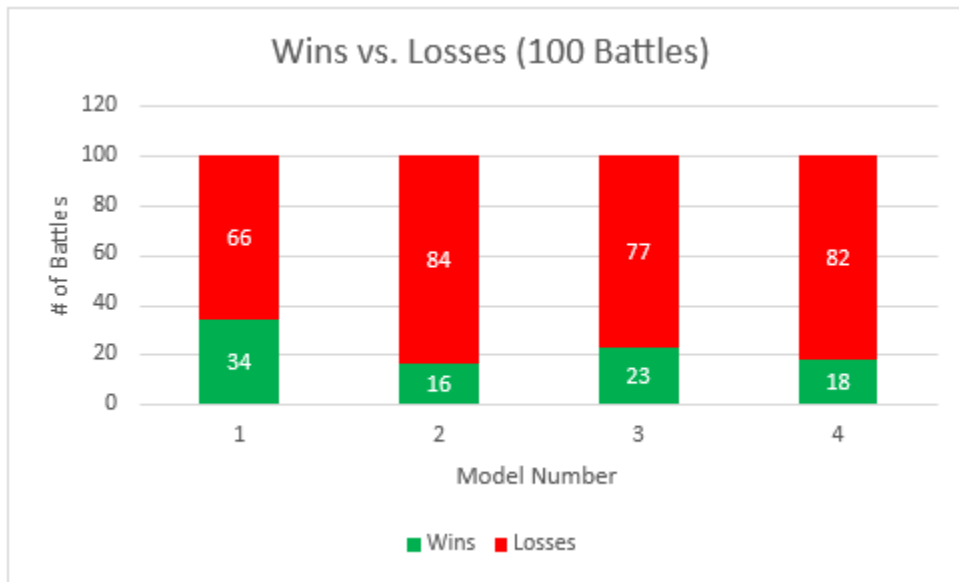


Figure 2: Visualization of Wins and Losses by Model Number over 100 battles.

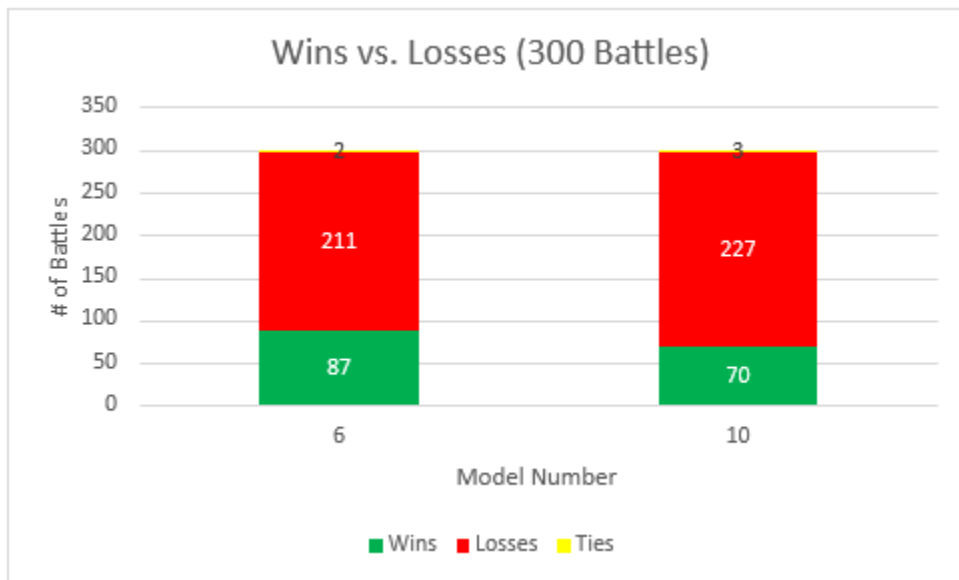


Figure 3: Visualization of Wins and Losses by Model Number over 300 battles.

Discussion and Insights

The initial win rate of 34% in Model 1 indicates that the rule-based heuristics provided a solid foundation for gameplay. The drop to 16% in Model 2 highlights the steep learning curve for the reinforcement learning structure, where the model needed to adjust for a vast set of rules before being able to perform well on its own. Part of the difficulty lies in the significant level of variance in Gen1 Randbats due to the randomized teams, which means the model can't generalize and learn the best way to play its own team since it does not remain consistent.

Another big hurdle was managing the random rolls in damage calculation and the non-determinism and hidden information in general. Since the model does not accurately know the

opponents' team composition at the start of the battle, there is a lot of prediction and learning that needs to come to make optimal decisions without perfect information. Certain moves may be optimal if they kill with certainty, but with the random rolls, there is the case that a move only has a chance of landing a KO, further convoluted by other factors such as accuracy.

We additionally faced several connectivity issues between Pokélearn and the Pokémon Showdown servers, including network restarts and constant crashes. This affected training cycles and created issues with a few models, hence the jump from Model 4 to 6, and from Model 6 to 10. To fix these issues, we implemented the following: queue and reconnect hardening, which checks connection health before deciding how aggressive recovery should be. Battle request deadlock protection was made with a timeout fallback to the default order so a missed pending order cannot block forever. If a battle request signature remains unchanged for too long, runtime retries order submission instead of stalling indefinitely. Timers are now enabled at the start of each battle, with runtime also using a safe, rate limited timer during active battles. Lastly, evaluation now uses the same core stall and requeue protections as heuristic and model collection. These fixes allowed long, overnight runs to be significantly more robust across server restarts, stuck queue/search states, and occasional request/order misses.

Due to the initial server issues, the model initially performed training in batches of 100 battles so that the crashes would not affect collecting training data. However, after the analysis of the initial training results, it became apparent that the training data was ineffective, primarily due to the search depth being too heavily restricted. This led to another separate training data set being created, which instead began training in batches of 300 battles since the crashing issue was no longer a limiting factor.

One final element of concern going into this project was that our work may be used for unintended purposes, such as cheating in the game. When considering the development of the UI, this scenario was kept in mind, and the system was designed towards being an educational tool to learn how to play Pokémon. In the context of competitive gaming online, there is a divide between whether it's ethical to use an advanced tool. In a lot of cases, it's considered an unfair advantage. According to Pokémon Showdown and the community, cheating usually falls into three main categories:

1. Automation: Running a script that plays thousands of matches with a "bot" that plays without human presence.
2. Ghosting: A high-level player telling a low-level player what to do during a tournament match.
3. Data Mining: Using exploits to see information that should be hidden information at the start of a match.

Since the deliverable project is a recommender system, it avoids the automation issue altogether, since user input and decision making is still required. Additionally, the recommender explains why a move is beneficial, which further pushes its educational purpose. The agent also only uses revealed information available to a normal player to avoid having an extreme advantage.

Future Work

Future extensions of Pokélearn include continued training over the next few months, outside of the academic term, in order to improve the model's win rate. Since Pokélearn has had a limited amount of time to train during the semester, extending the training time would expose the model to a significant number of battles. This will assist with the learning process developing new battle strategies..

Additionally, Pokélearn can explore other battle formats, such as random battles with no generation limitations or random doubles battles. This would allow the model to learn and utilize newer game mechanics, such as Generation 6's Mega Evolution, which drastically increases a Pokémon's

strength via base stats, or Generation 7's Z-Moves, which introduce single-use battle moves. Additionally, battle formats such as doubles (2v2s), would force the model to learn how to adapt to battle settings that require more Pokémon on the field.

Lastly, Pokélearn can incorporate battle-specific features, such as held items, into training. Pokémon are capable of holding items that provide additional perks. Each item has different effects, with some affecting the Pokémon's status conditions, boosting stats, or powering up moves. Introducing held items would add another layer of complexity of learning to the model, allowing for more possible tactics.

Conclusion

We successfully created an educational recommender system that considers user input and decision making while avoiding creating unfair advantages. Instead of being a fully automated AI agent, Pokélearn was designed to work alongside the pilot player through move recommendations and reasoning based on information collected during battles. This further emphasized Pokélearn's intentions of being a learning tool.

Additionally, we learned what moves are optimal during a given battle based on model reasoning. The heuristics, shallow search, and policy-value network assisted in making solid decisions across diverse battles. However, human players often have stronger strategic reasoning due to battle experience, demonstrating the gap between learned decision-making and gained human experience.

Hence, Pokélearn shows how learning systems can be used as an educational tool that allows players to learn and make better decisions in battle, while also leaving room for improvements in terms of performance.

References

- n.d. Pokémon Showdown! battle simulator. Accessed April 29, 2026. <https://pokemonshowdown.com>.
- “Damage - Bulbapedia, the community-driven Pokémon encyclopedia.” n.d. Bulbapedia. Accessed April 29, 2026. <https://bulbapedia.bulbagarden.net/wiki/Damage>.
- Flaherty, Joseph, Aaron Jimenez, and Bahareh Abbasi. 2023. “Playing Pokemon Red with Reinforcement Learning.” *Computer Science Conference for CSU Undergraduates*, (March), 6. <https://scholarworks.calstate.edu/downloads/vq27zt20r>.
- “Formats | RB.” n.d. Smogon. Accessed April 29, 2026. <https://www.smogon.com/dex/rb/formats/>.
- Hu, Sihao, Tiansheng Huang, and Ling Liu. 2024. “PokéLLMon: A Human-Parity Agent for Pokémon Battles with Large Language Models.” *arXiv* 3 (April): 10. <https://arxiv.org/pdf/2402.01118>.
- Karten, Seth, Andy L. Nguyen, and Chi Jin. 2025. “PokéChamp: an Expert-level Minimax Language Agent.” *arXiv* 1 (March): 25. <https://arxiv.org/pdf/2503.04094>.
- “Poke-Env.” n.d. Poke-env: A Python Interface for Training Reinforcement Learning Pokémon Bots. Accessed April 29, 2026. <https://poke-env.readthedocs.io/en/stable/index.html#>.
- “Pokémon Damage Calculator.” n.d. Pokémon Showdown. Accessed April 29, 2026. <https://calc.pokemonshowdown.com/randoms.html?gen=1&mode=randoms>.
- “Pokémon moves from Generation 1 | Pokémon Database.” n.d. Pokemon Database. Accessed April 29, 2026. <https://pokedexdb.net/move/generation/1>.
- “Pokémon type chart: strengths and weaknesses | Pokémon Database.” n.d. Pokemon Database. Accessed April 29, 2026. <https://pokedexdb.net/type>.
- “Stat - Bulbapedia, the community-driven Pokémon encyclopedia.” n.d. Bulbapedia. Accessed April 29, 2026. <https://bulbapedia.bulbagarden.net/wiki/Stat>.